IFPI
Campus
Floriano

Aula 7: Herança

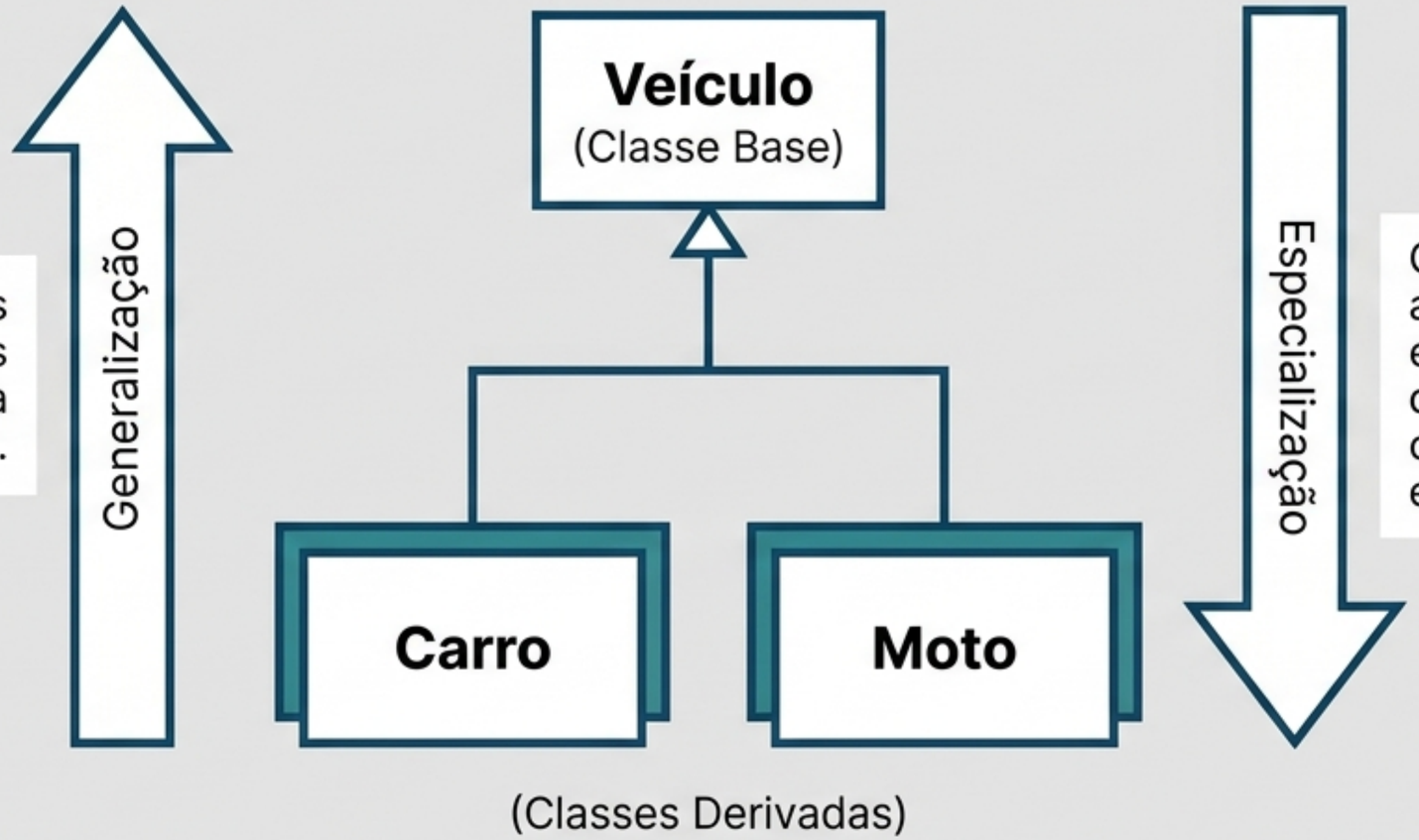
Programação Orientada a Objetos com TypeScript

Prof. Ronaldo Borges

Gemini Notebook

A Essência da Herança

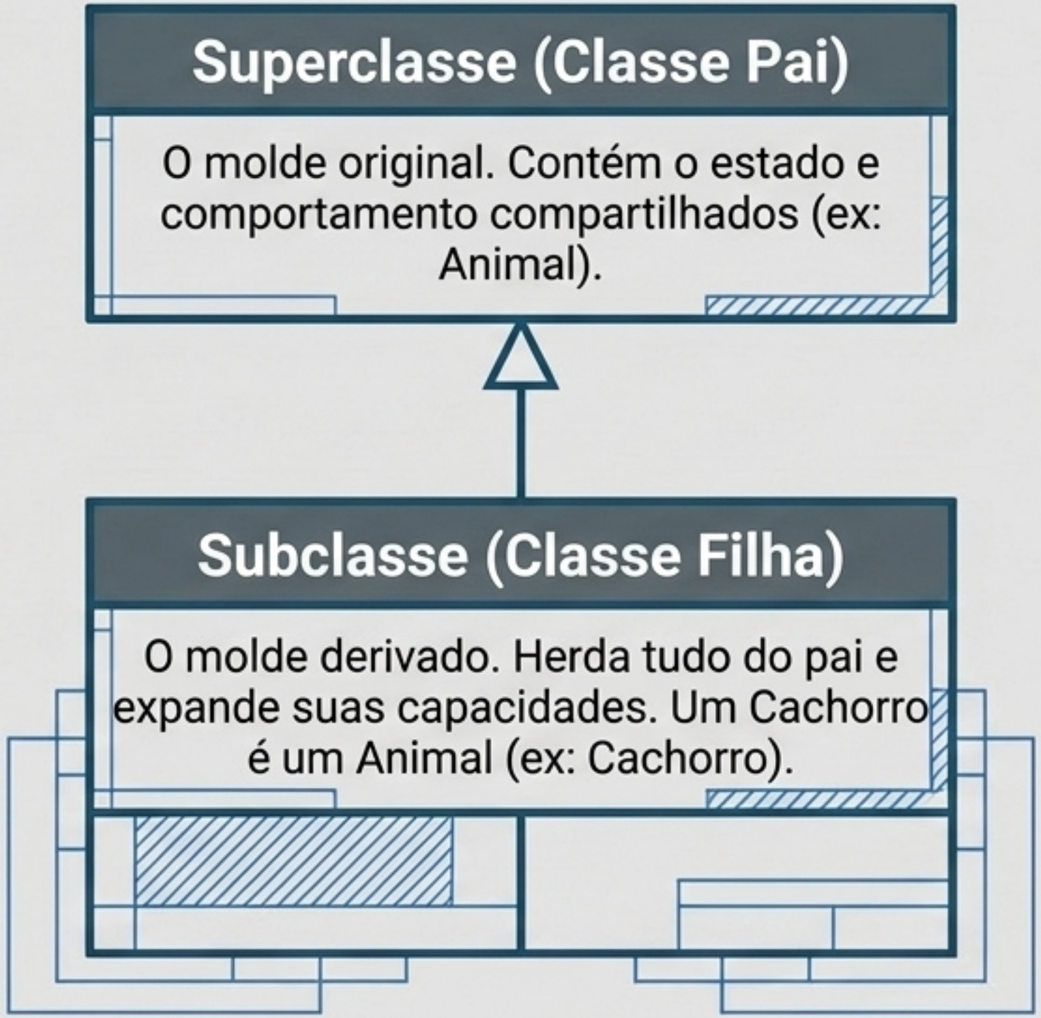
Extrair características comuns de várias classes para formar uma única classe base abstrata.



Criar uma nova classe a partir de uma existente, adicionando detalhes e comportamentos específicos.

Gemini Notebook

Anatomia da Hierarquia



Gemini Notebook

A Ponte Estrutural: extends

```
class Pessoa {
    nome: string;
    falar() { return `${this.nome} está falando.`; }
}

class Vendedor extends Pessoa {
    totalVendas: number = 0;
}
```

A Palavra-Chave: Estabelece o vínculo evolutivo. A classe filha absorve automaticamente propriedades e métodos da classe pai.

Absorção Silenciosa: O Vendedor agora possui .nome e .falar(), mesmo sem reescrevê-los.

Gemini Notebook

A Regra de Ouro: O uso do super()

```
class Carro extends Veiculo {
  constructor(marca: string, numPortas: number) {
    super(marca); // Obrigatório!
    this.numPortas = numPortas;
  }
}
```

1

Aviso Crítico: Construtores em classes derivadas exigem a chamada de super().

2

A Lógica: O molde pai deve ser construído e inicializado primeiro.

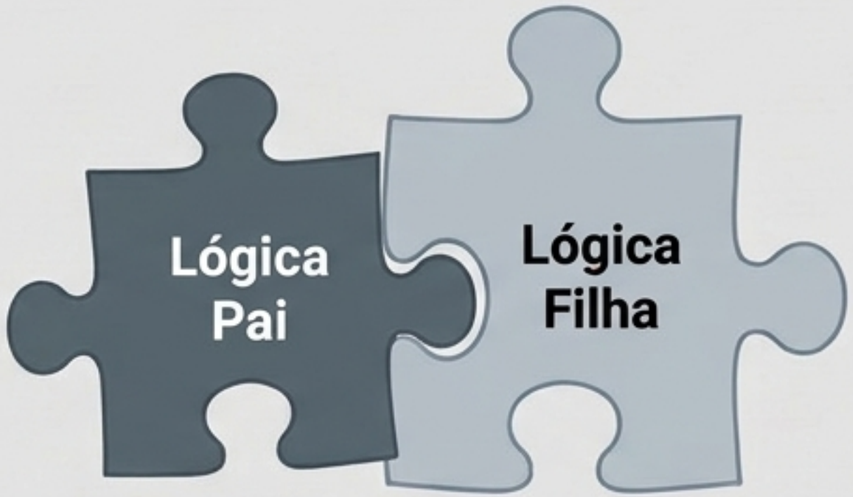
3

A Consequência: É impossível usar a referência **this** na subclasse antes que super() seja invocado.

Gemini Notebook

Reaproveitando Lógica com super.metodo()

```
class Gerente extends Funcionario {
  calcularBonus(): number {
    // Reaproveita a regra base e adiciona R$ 500
    return super.calcularBonus() + 500;
  }
}
```



O ***super*** não é exclusivo do construtor. Ele atua como um ponteiro direto para a classe base.

Uso: Permite invocar a versão original de um método herdado de dentro da subclasse, expandindo a lógica em vez de descartá-la.

Gemini Notebook

O Dilema do Encapsulamento

`public`



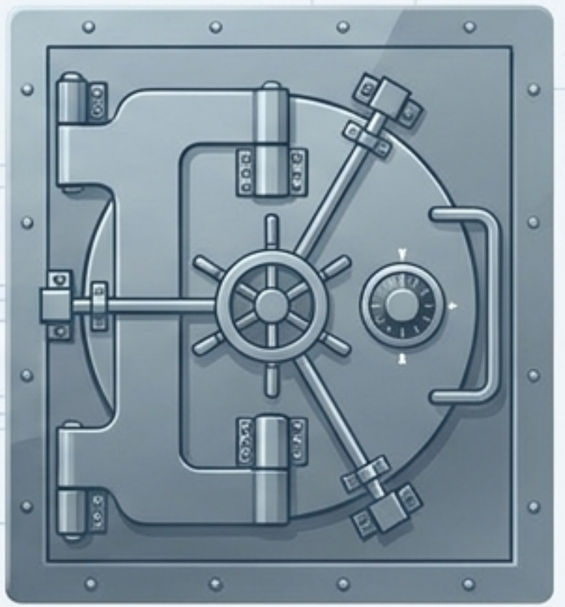
Acesso livre para qualquer parte do sistema.

`protected`



O meio-termo cirúrgico. Restrito ao público, mas acessível à família.

`private`



Se a classe filha herda, ela não pode tocar. Cofre fechado.

Gemini Notebook

Matriz de Visibilidade

Modificador	Acesso na Própria Classe	Acesso nas Subclasses	Acesso pelo Mundo Externo
public	✓	✓	✓
protected	✓	✓	✗
private	✓	✗	✗

O verdadeiro propósito do protected: Proibir o acesso de instâncias externas, mas garantir que as subclasses filhas possam ler e alterar o membro herdado. É um segredo de família.

Gemini Notebook

protected em Ação

Acesso Válido



```
class Conta {
  protected saldo: number = 0;
}

class ContaCorrente extends Conta {
  sacar(valor: number) {
    this.saldo -= valor; // PERMITIDO
  }
}
```

Erro de Compilação



```
const minhaConta = new ContaCorrente();

minhaConta.saldo = 1000;

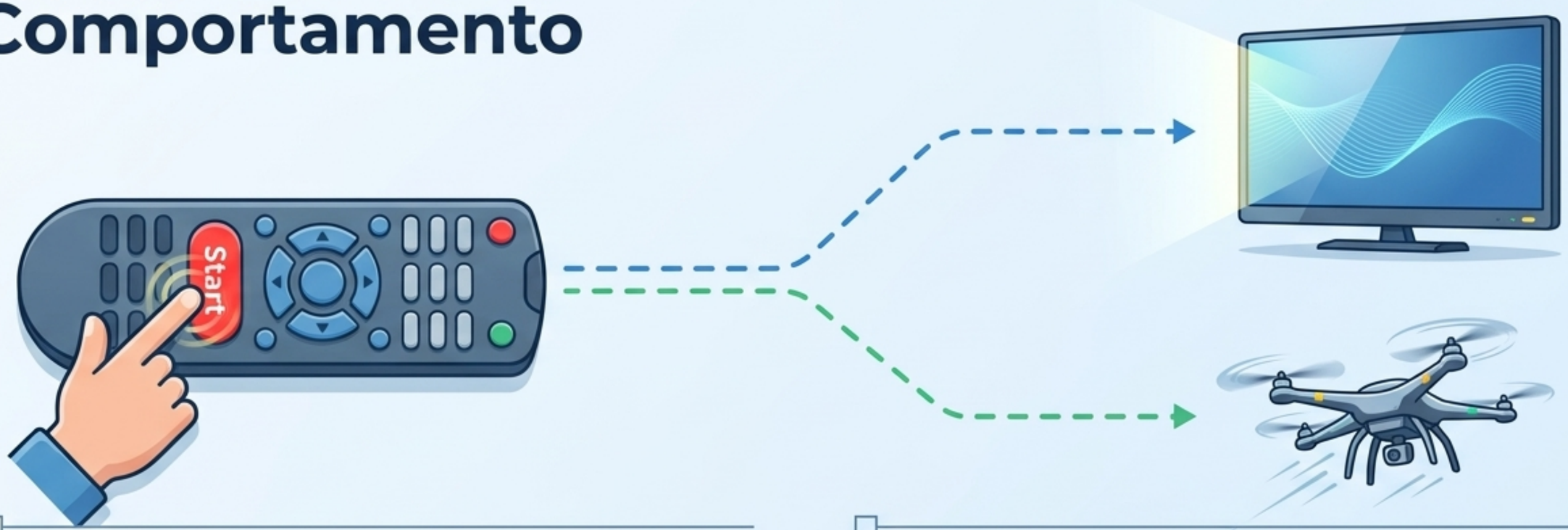
// ERRO: A propriedade 'saldo' é protegida
// e só pode ser acessada dentro da classe
// 'Conta' e de suas subclasses.
```



// ERRO: A propriedade 'saldo' é protegida
// e só pode ser acessada dentro da classe
// 'Conta' e de suas subclasses.

Gemini Notebook

Introdução à Mutação de Comportamento



Polimorfismo:

Herança não é apenas sobre reaproveitar atributos passivos. É sobre adaptar comportamentos para contextos específicos.

Duas vias principais:

Existem duas formas de alterar como métodos reagem no sistema: Modificar a essência do método herdado (Override) ou oferecer múltiplas formas de invocação (Overload).

Gemini Notebook

Sobrescrita de Métodos (Override)

```
class Animal {
  mover() { console.log("O animal está se movendo."); }
}

class Sapo extends Animal {
  mover() { console.log("O sapo está pulando."); }
}
```

Definição:

Mudar a implementação interna de um método herdado.

A Regra:

A assinatura (nome e parâmetros) permanece a mesma, mas o comportamento interno é substituído na classe filhas filha. O método original é reescrito.

Gemini Notebook

Sobrecarga de Métodos (Overload)



```
class Livro {
  // Múltiplas Assinaturas (Declarações)
  constructor(titulo: string);
  constructor(titulo: string, autor: string);

  // Única Implementação Real
  constructor(titulo: string, autor?: string) {
    // Lógica interna checa os tipos/argumentos
  }
}
```

Definição:

O mesmo método oferecendo múltiplas assinaturas de parâmetros (quantidade ou tipos).

Peculiaridade TypeScript:

Devido ao JavaScript subjacente, provemos múltiplas assinaturas teóricas, mas escrevemos apenas uma implementação real flexível.

Gemini Notebook

Matriz de Diagnóstico: Override vs. Overload

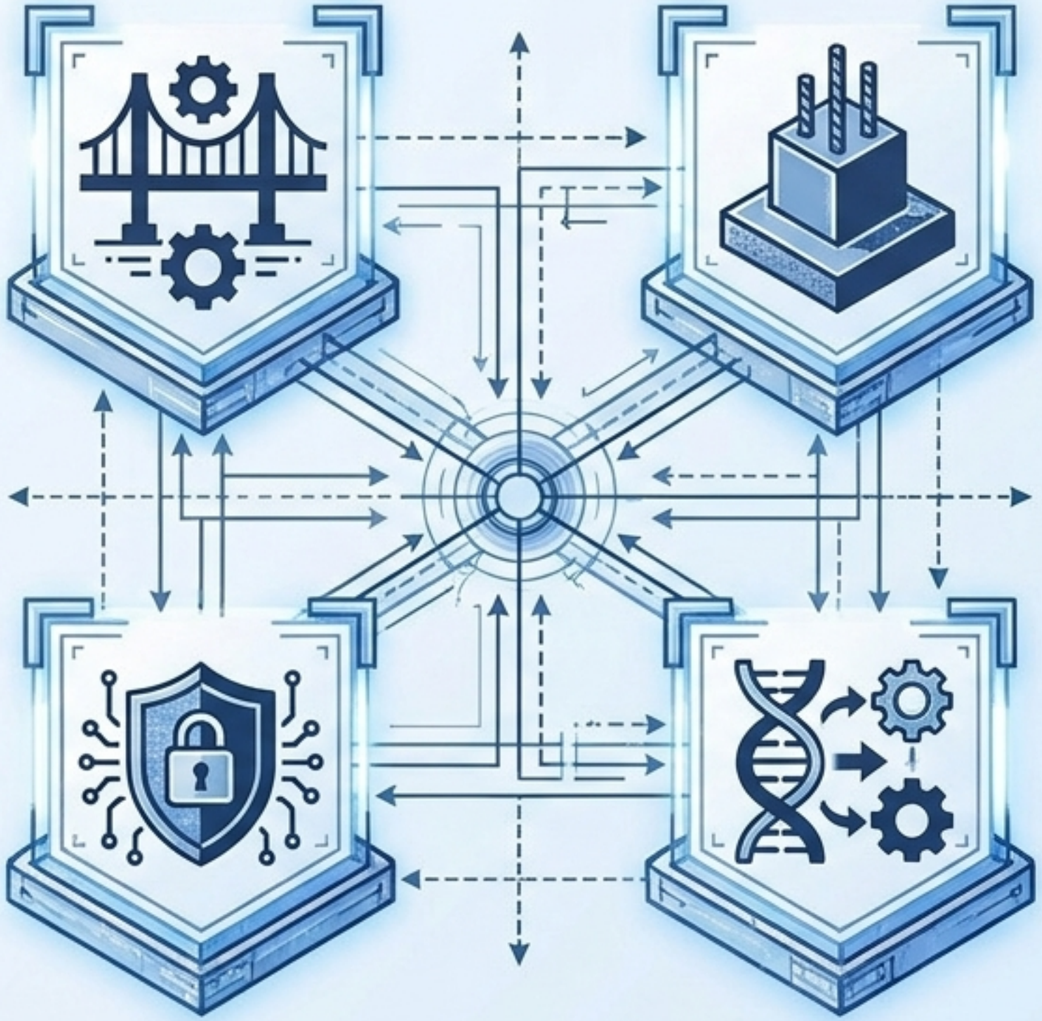
Critério	Override (Sobrescrita)	Overload (Sobrecarga)
Onde ocorre?	Entre Superclasse e Subclasse (Hierarquia)	Na mesma classe (ou cadeia de herança)
Assinatura	Exatamente a mesma (mesmos parâmetros)	Deve ser diferente (quantidade ou tipos de parâmetros)
Propósito principal	Especializar ou substituir um comportamento genérico do pai	Oferecer flexibilidade de argumentos para a mesma ação

Gemini Notebook

O Mapa da Herança

A Estrutura (extends)

A ponte que permite que a subclasse absorva o molde da superclasse.



A Fundação (super())

O chamado obrigatório que garante que o objeto pai ganhe vida antes do filho.

A Segurança (protected)

O guardião que oculta os dados do mundo, mas os confia à linhagem familiar.

A Evolução (Override/Overload)

O mecanismo de mutação que permite aos objetos adaptarem seus comportamentos às novas novas necessidades.

Gemini Notebook

Referências Bibliográficas

ADRIANO, da Silva Adriano. Guia prático de TypeScript: Melhore suas aplicações JavaScript. São Paulo: Casa do Código, 2021.

MICROSOFT. TypeScript Language Specification. Versão 0.9.1. 2013.

SCHILDT, Herbert. Java para iniciantes: crie, compile e execute programas Java rapidamente. 6. ed. Porto Alegre: Bookman, 2015. 684 p.

BORGES, Ronaldo. Notas de Aula: Estruturas Sequenciais e Orientação a Objetos. IFPI, 2025.

Gemini Notebook